# E-Deformable Gaussians General

Target: Monocular, camera completely fixed with no displacement, event stream input-only, 4DGS

Lee Minghao (Aiur)
2330034027

# Original Experiment

/review/das3r-dynamics-aware-gaussian-splatting-for-static-scene-reconstruction

During Experiment, we have found some pain points:

– 4DGS Assumption Limitation: translational/rotational parallax and highly relies on multi-views positions

– Wrong depth ambiguity: Fixed cameras have no spatial parallax, lead to geometric degradation in purely temporal observations and strong depth ambiguity;

– Static/Dynamic Gaussian Non-Separation: static=90%, and current methods NO separate static/dynamic Gaussians, relying entirely on deformable fields, leading to computational overload and severe noise;

– Event-Only Limitation: events only record brightness changes, and fixed viewpoints lack spatial gradients

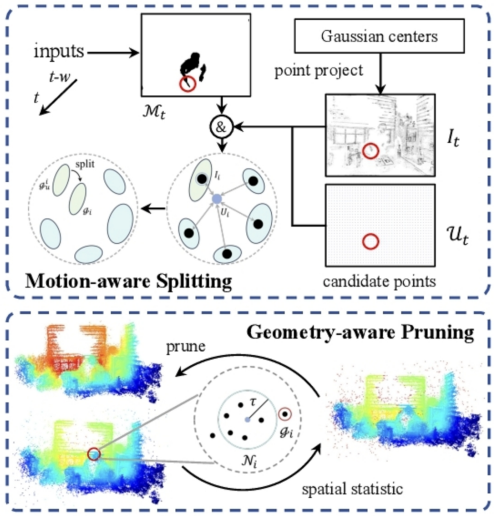
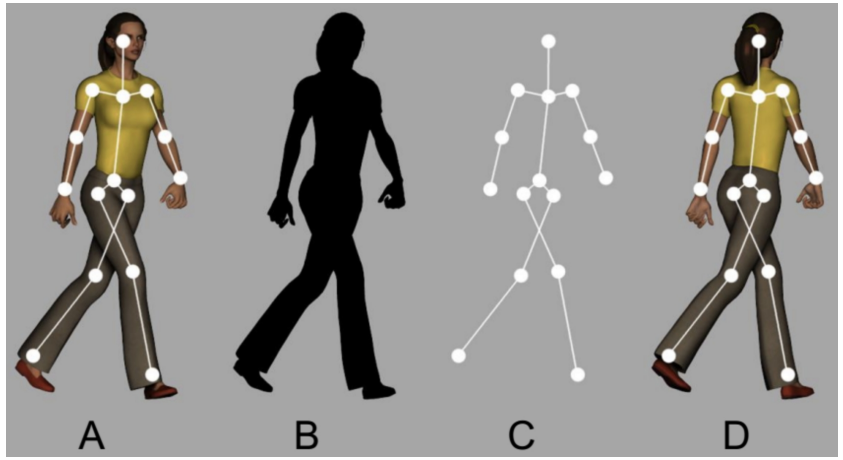
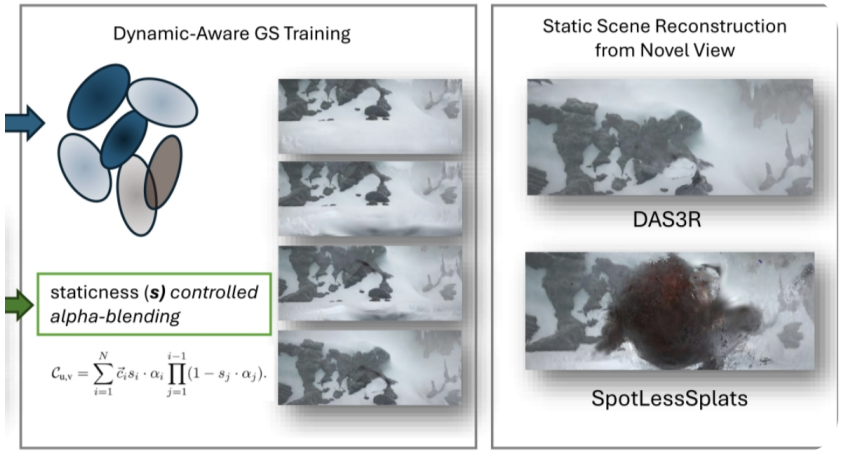



Figure 3: Illustration of Spatial-temporal Density Control strategy.




(2) depth ambiguity problems




(3) Static/Dyns GS Non-Separated

# Innovation 1: Static Background Decoupling + Temporal Optical Flow–Driven Dual–Branch 4D Gaussian Represents

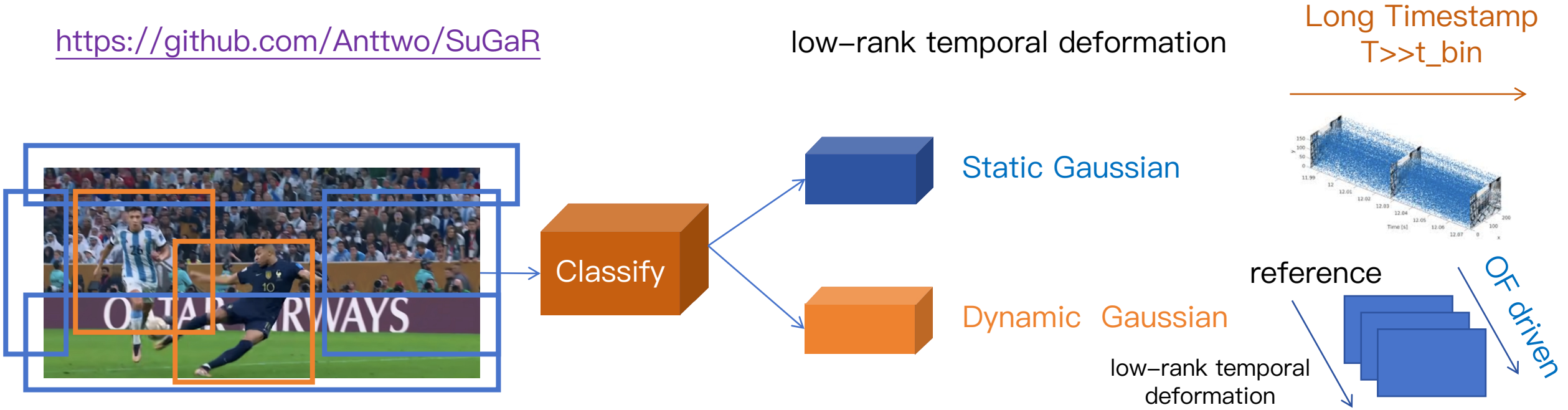
Splits a permanent static Gaussian (BG) + a temporal dynamic Gaussian (FG + Moving);

– The static Gaussian disables the deformation network, using only long–term event accumulation for geometric refinement, permanently fixing the spatial position;

The dynamic Gaussian is driven by event–asynchronous optical flow to get a low–rank temporal deformation basis, eliminating the need for frame–by–frame displacement optimization;

https://github.com/Anttwo/SuGaR

low–rank temporal deformation



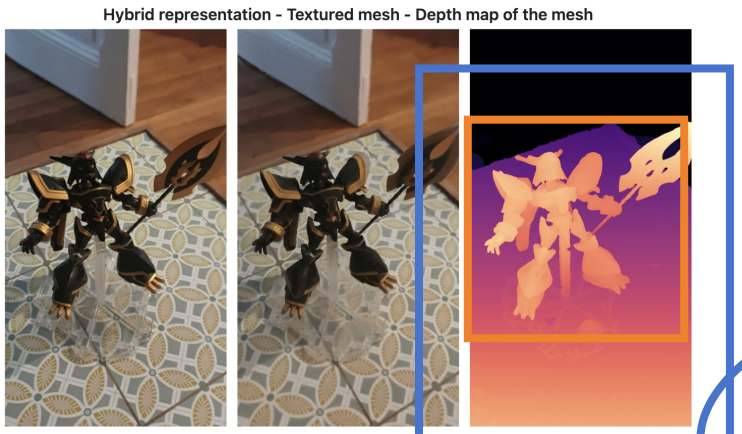
# Innovation 1: Dual–Branch 4D Gaussian Represents

We Design the Loss (2 parts):

- Static region event consistency regularization to suppress background Gaussian drift;
- Dynamic region event temporal differential constraint.

$$\mathcal{L} = \underbrace{\|E - \hat{E}\|_1}_{event reconstruction} + \lambda_s \underbrace{\sum M_s \|\theta(t) - \theta(t_0)\|^2}_{static consistency} + \lambda_d \underbrace{\sum M_d \left\| \frac{\partial \hat{I}}{\partial t} - E \right\|_1}_{dynamic temporal constraint}$$

$$\mathcal{L} = \mathcal{L}_{event} + \lambda_1 \mathcal{L}_{static_reg} + \lambda_2 \mathcal{L}_{dynamic_temp}$$




$$\mathcal{L}_{static} = \sum_{x,t} M_s(x) \cdot \|\hat{E}(x,t)\|_1$$

$$\mathcal{L}_{static} = \sum_{i \in \text{static}} \|\theta_i(t) - \theta_i(t_0)\|_2^2$$

$$\mathcal{L}_{dynamic} = \left\| \frac{\partial \hat{I}(x,t)}{\partial t} - \hat{E}(x,t) \right\|_1$$

$$\mathcal{L}_{dynamic} = \sum_x M_d(x) \cdot \|\theta(x, t+\Delta t) - \theta(x,t)\|_2^2$$

Static

Dynamic

Static

Dynamic

GSs θ radiance/depth should NOT "drift"

+ depth of obj NOT change (e=0, black)

Time Smoothing event–based temporal derivative matching

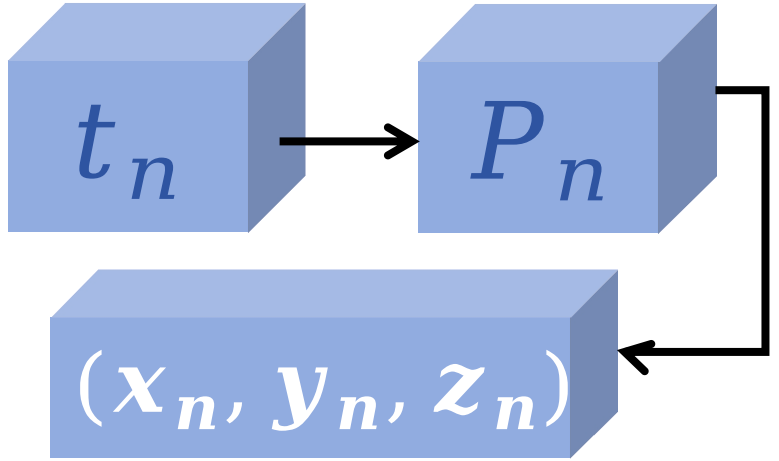
# Innovation 2: Fixed–Pose Temporal Stereo parallax modeling

The camera remains stationary (monocular fixed), while the object moves,

– using different times $t_1, t_2, t_3 \ldots t_n$ as "different perspectives"

| Traditional Stereo | Ours |
|---|---|
| left / right | $t_1, t_2, t_3 \ldots t_n$ |
| Spatial baseline | Temporal baseline |

One–to–One Pairs








$\boldsymbol{Camera\ Poses\ P_1\ (x_1, y_1, z_1)}$

$\boldsymbol{P_n\ (x_n, y_n, z_n)}$

$\boldsymbol{\ldots}$

$\boldsymbol{P_3\ (x_3, y_3, z_3)}$

$\boldsymbol{P_2\ (x_2, y_2, z_2)}$

t process....

$\boldsymbol{t_1}$ $\boldsymbol{t_2}$

# Innovation 3: Gaussian Space Adaptive Event Threshold

$$|\Delta \log I(x,t)| > C$$

Each 3D Gaussian (point) is bound to a threshold:
Ci for each Gaussian i, C=C(Gaussian index)

Scenes Threshold Presentations:

SAM2 to split out target object




Frame Version

| Gaussian Object | Threshold |
|---|---|
| Static Background | High (–insensitive) |
| Moving Object | Low (–sensitive) |
| Edge Area | Medium |

## GS–Threshold Co–optimization

Joint optim: $\theta_{gaussian}, C_i$

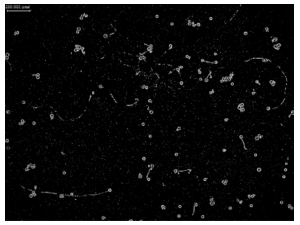
$$\mathcal{L} = \mathcal{L}_{render} + \mathcal{L}_{event}(C_i) + \mathcal{L}_{reg}(C_i)$$

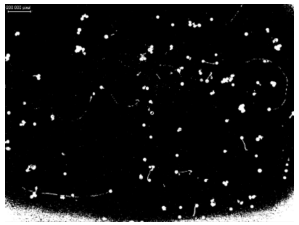
- Low GSs Density Distribution
- High GSs Density Distribution
- Medium GSs Density Distribution

Learnable !

$$E(x,t) = 1(|\Delta I(x,t)| > C_i)$$

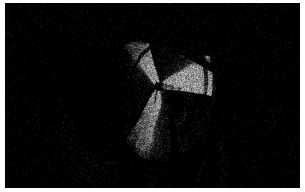
# Scenes Threshold Presentations:

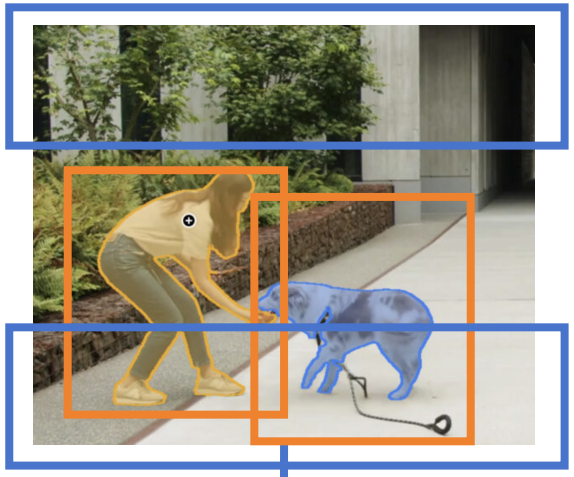




similar effects with
denoising algorithm,
but motion–targeted








$\theta_{static-t}$
$(gaussian-static)$

$\theta_{dynamics-t}$
$(gaussian-dynamics)$

| Gaussian Object | Threshold |
|---|---|
| Static Background | High (–insensitive) |
| Moving Object | Low (–sensitive) |
| Edge Area | Medium |

tips: Although SAM2 model segementation makes “people” and “dog” into different categories, but in our model, we all viewed them as “dynamic” state of Gaussians, no matter what object it’s.



Question: we will later do some expriments to decide which design is better for filtering out “static” or “dynamics” state for GSs:
– move / or not
– setting with certain thresholds (depends on scenes)

# Innovation 4: Use events to monitor t–derivative of Gaussian

~~Event → Image/Frame → Retraining (Lossy Compression)~~

Lossy!

$$E(x,t) \propto \frac{\partial I(x,t)}{\partial t}$$

Event Trigger!

$$\mathcal{L} = \sum_k \left| \frac{d}{dt} \log I(x_k, t_k) - p_k \cdot C \right|$$

Accumlation

By directly utilizing the original asynchronous event points and through the consistency constraint between the time derivative of Gaussian rendering brightness and the event triggering conditions, continuous–time 4DGSs supervised learning without frame breaks or E2VID is achieved.

# Training pipeline engineering: Three–stage training

1) Static background pre–reconstruction:
Only the first 30 seconds of events are accumulated, freezing the background Gaussian;

2) Dynamic Gaussian incremental generation: Event stream is used to guide new Gaussians only in moving regions;

3) Global joint fine–tuning: Background is fixed, dynamic deformation field is finely optimized;

# Experiment 1: SAM2 + P-MASK (self-set model)

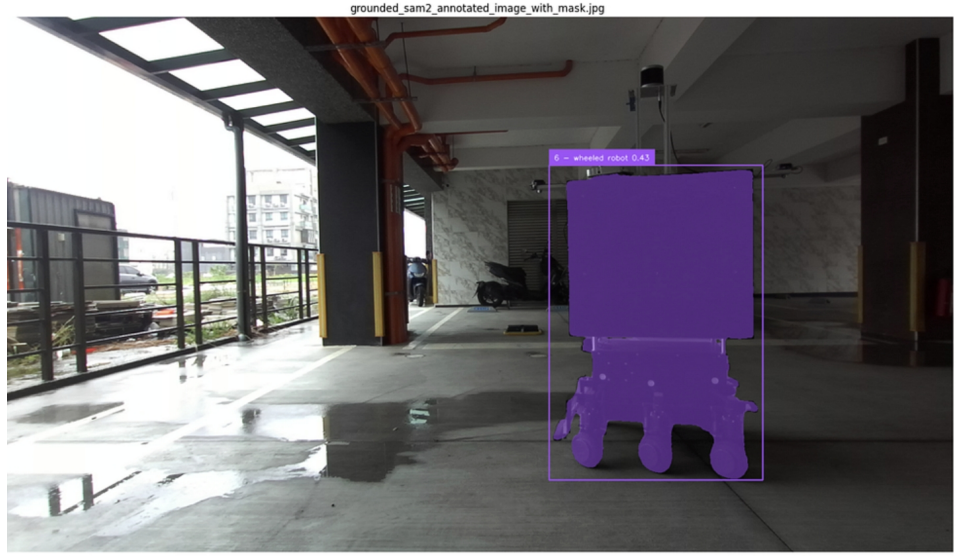
> SAM2: not support NLP and language input & must hand-in MASK for init frame to target object

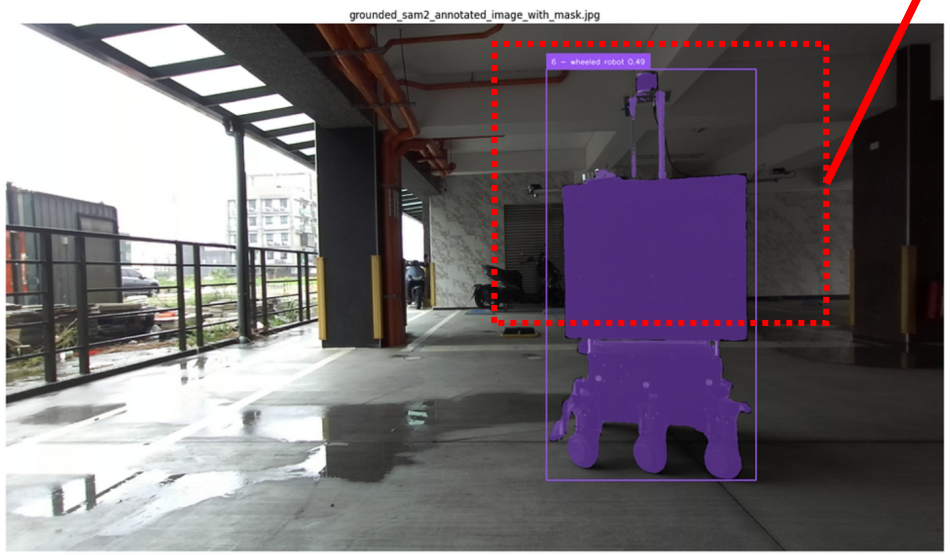
– attention drift problems! (especially for fast moving obj & Flash Frame) –-> fail to target obj

> P-MASK (self developed model): support NLP and language-guided obj targetting

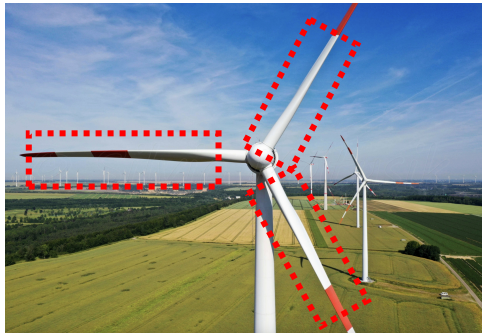
+ Solving memory bank problem caused by accumulated errors

6-wheeled robot vs 6-wheeled robot with LiDAR



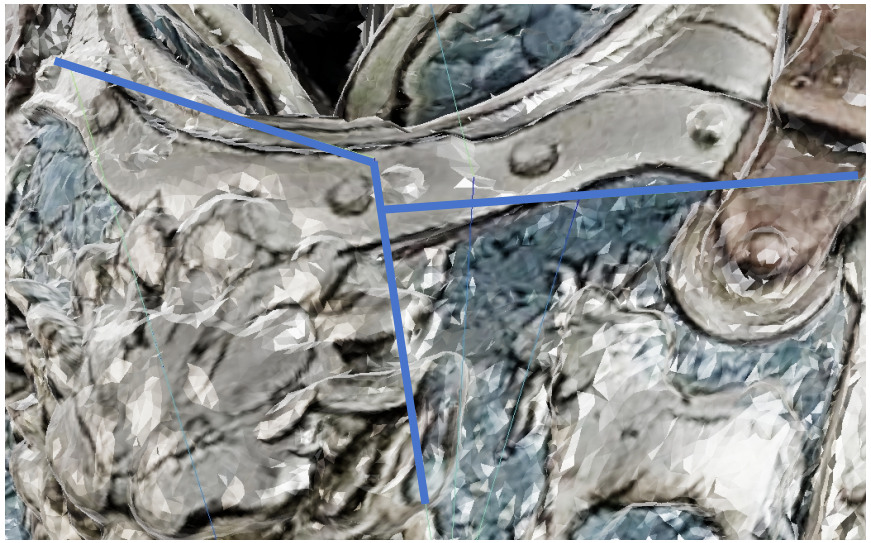

SAM2 can detect small, and elongated objects in the scene also be identified, such as wind turbine blades.



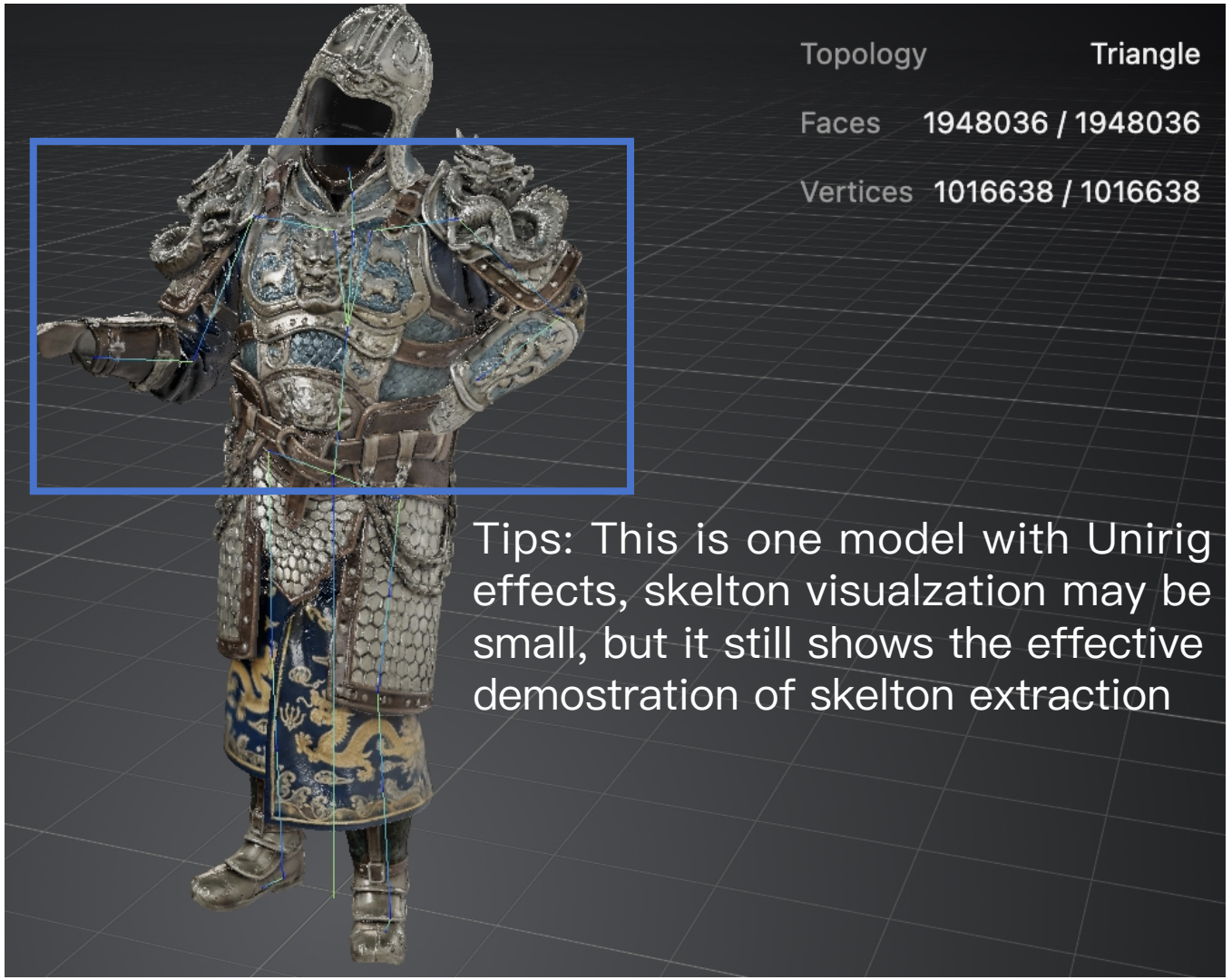
wind turbine blades

# Experiment 2: Unirig--Skelton Extraction



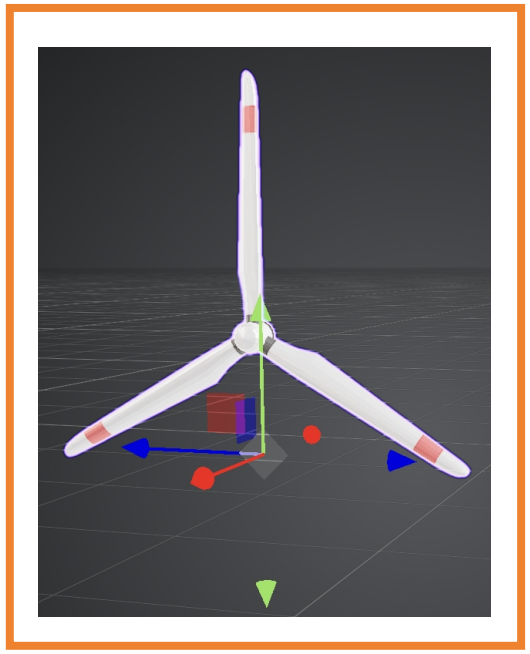


self experiment with armor model

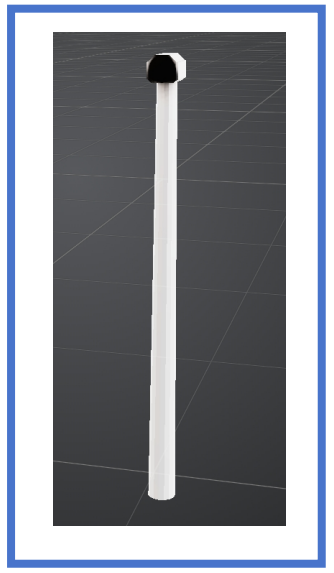




SAM2



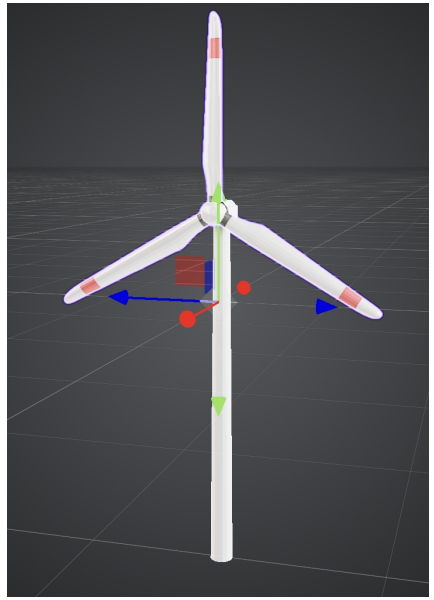
Dynamic



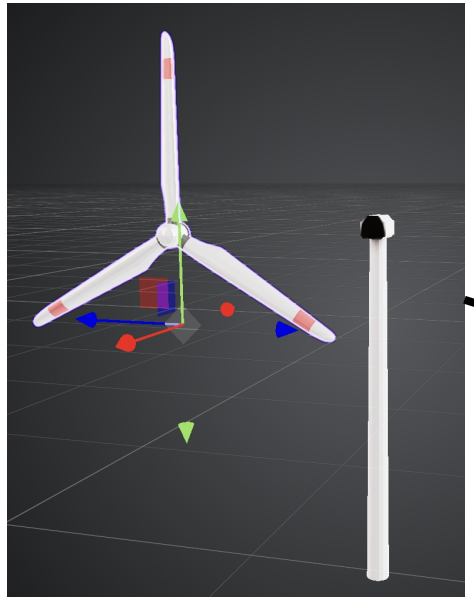
Static

Motion Detect

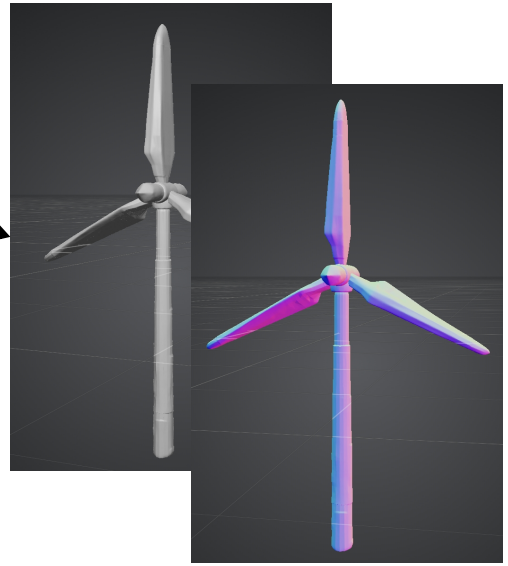
Only Filtering out the dynamic object = “Wind Tarbine”

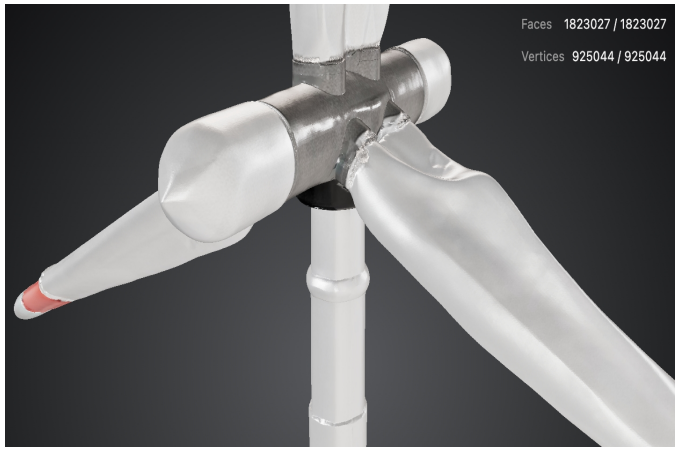


our init Wind Turbine model

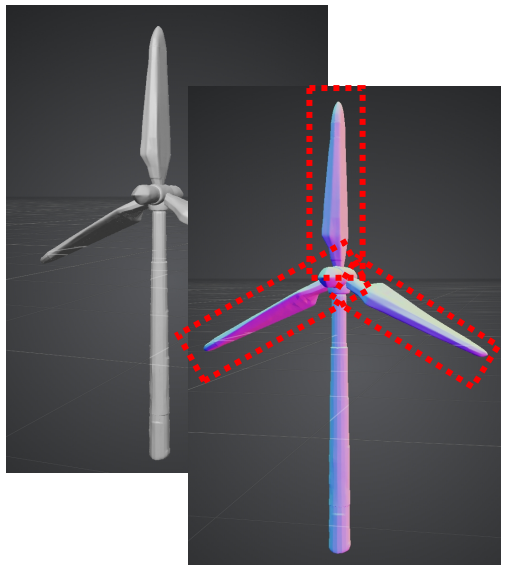


Divide into TWO parts





# Experiment 2–Failure: Skeleton Extraction Limit



After experiment, we know the rigging / skeleton extraction has applicable boundary, like the following scenes: (Skeleton Extraction fail!)

> Objects without a defined skeletal structure: These have limited animation potential, such as waves, water, and smoke.

> Highly branching, complex structures: Dense objects (trees, bushes,  NN).

> Objects with extremely unstable thickness variations & 2D–like meshes

> Highly rigid with no deformation requirements: Such as some industrial parts and mechanical structures; transform/rigid body is more suitable.

# Ours (Wind Turbine Application)

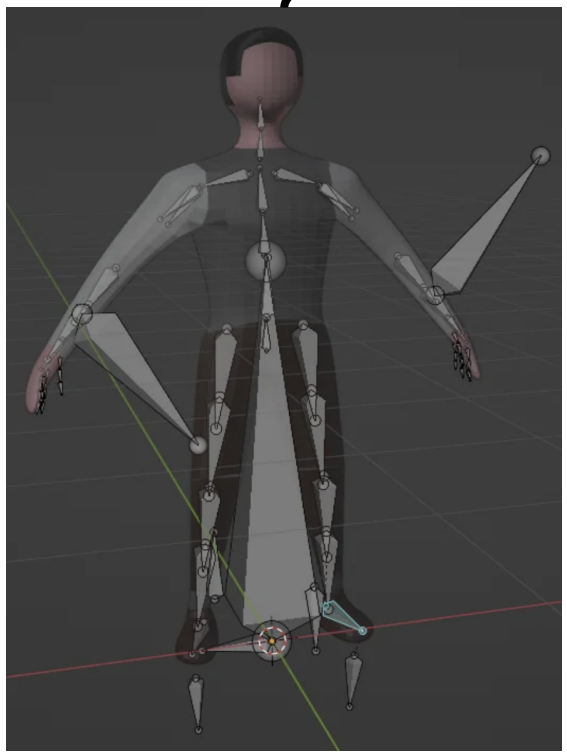
The blade is a thin sheet, not a chain structure, and is easily perceived by the model as "a whole fan blade viewed as a single part." This leads to a degeneration into a center point + three branches.

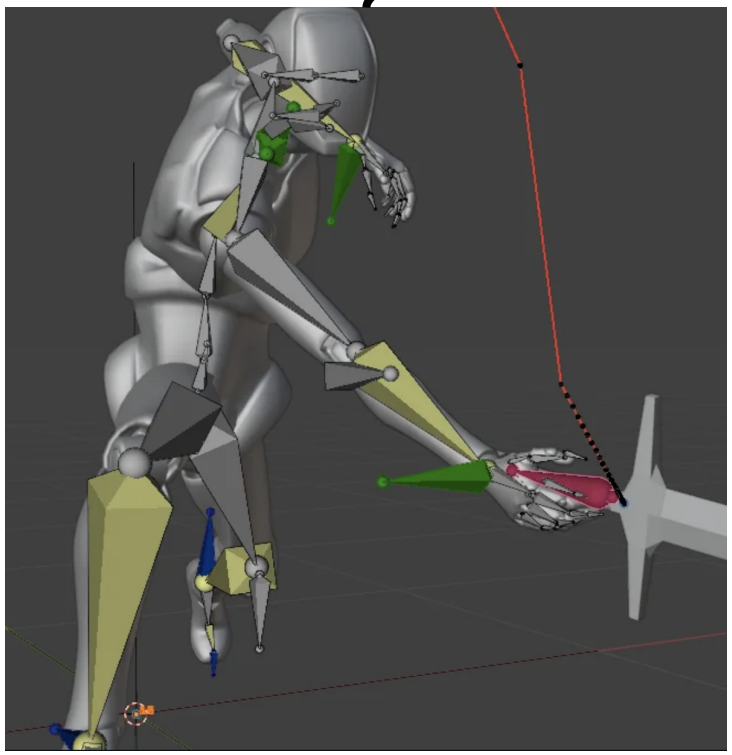
Specifically Rigging designed for T–pose or A–pose stances, it accurately identifies the positions of key joints such as the shoulders, elbows, hips, and knees, resembling the unfolded state of a fan blade.

# Ours (Wind Turbine Application)

The blade is a thin sheet, not a chain structure, and is easily perceived by the model as "a whole fan blade viewed as a single part." This leads to a degeneration into a center point + three branches.

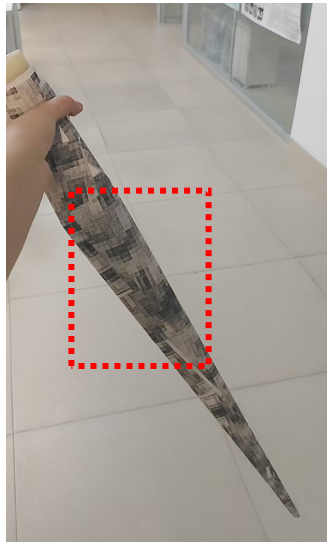
Specifically Rigging designed for T-pose or A-pose stances





Our experiments on A-pose and T-pose object detection

These two are better shaped in T-Pose and A-Pose

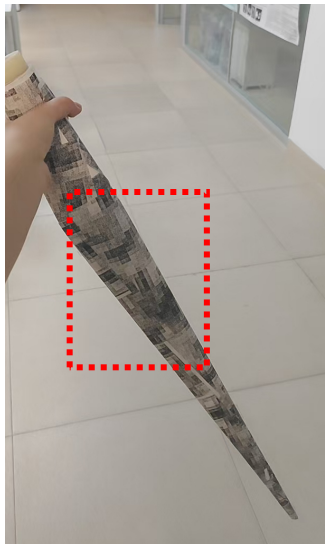


Hard and indeformable

This Wind Turbine Blade is thin and not a chain structure, let alone the T/A-Pose.



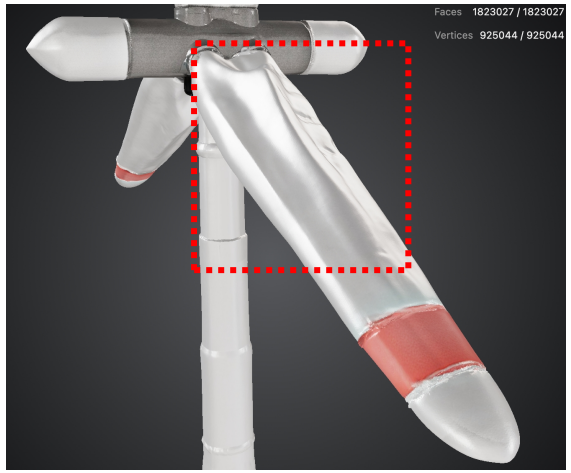
# Ours (Wind Turbine Pain Points*2)

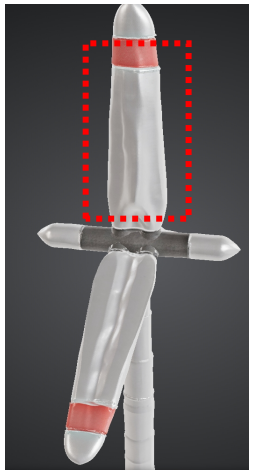






Wind Turbine
Side View



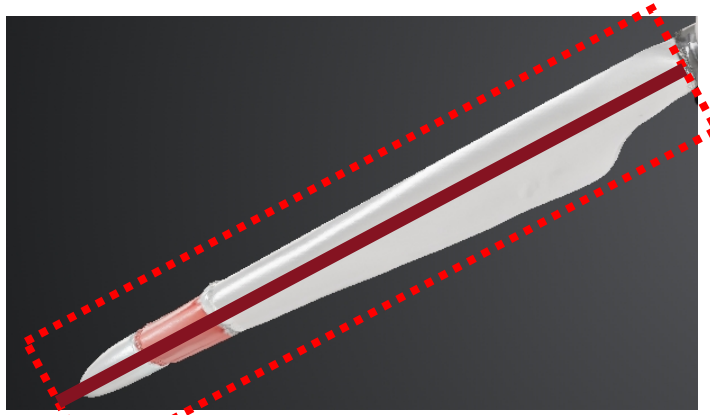
Wind Trubine Skelton Point 1:
Non-Smooth, Uneven Surface

Wind Trubine Skelton Point 2:
Non-node Chain Structure

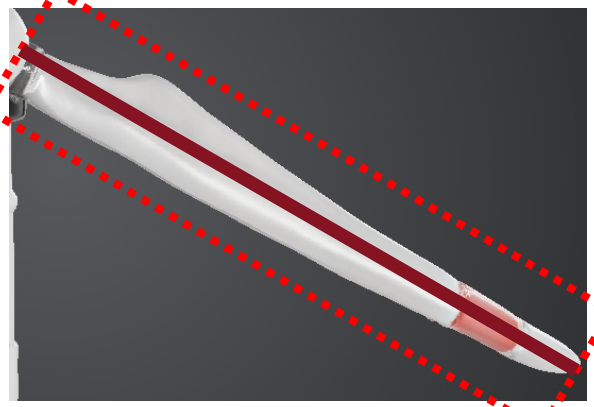
No-Need to Extract Backbone

The reconstructed surface is not ~~smooth or flat~~. Uneven surface, causing ~~abnormal detection~~ fail !

→ wrinkles are considered "abnormal" in nature



Blade 1

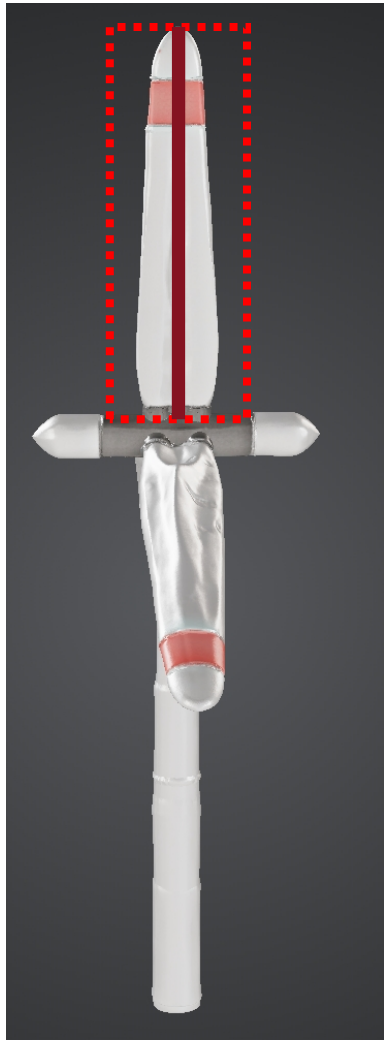


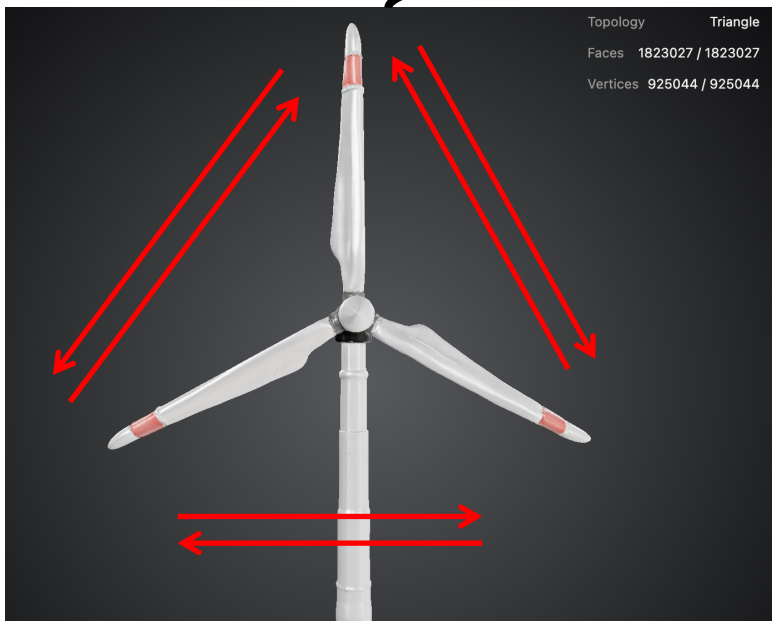
Blade 2



# Ours (Further into Wind Turbine Coupled Motion)

In fact, the division of wind turbine coupled motion is not based on the skeleton as a fulcrum.





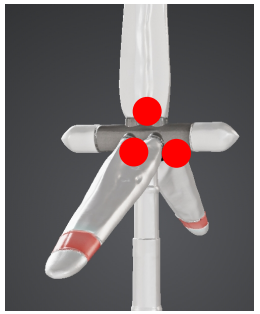

Motion 1: Frontal Planar Rotation



Motion 2: Side Planar Rotation

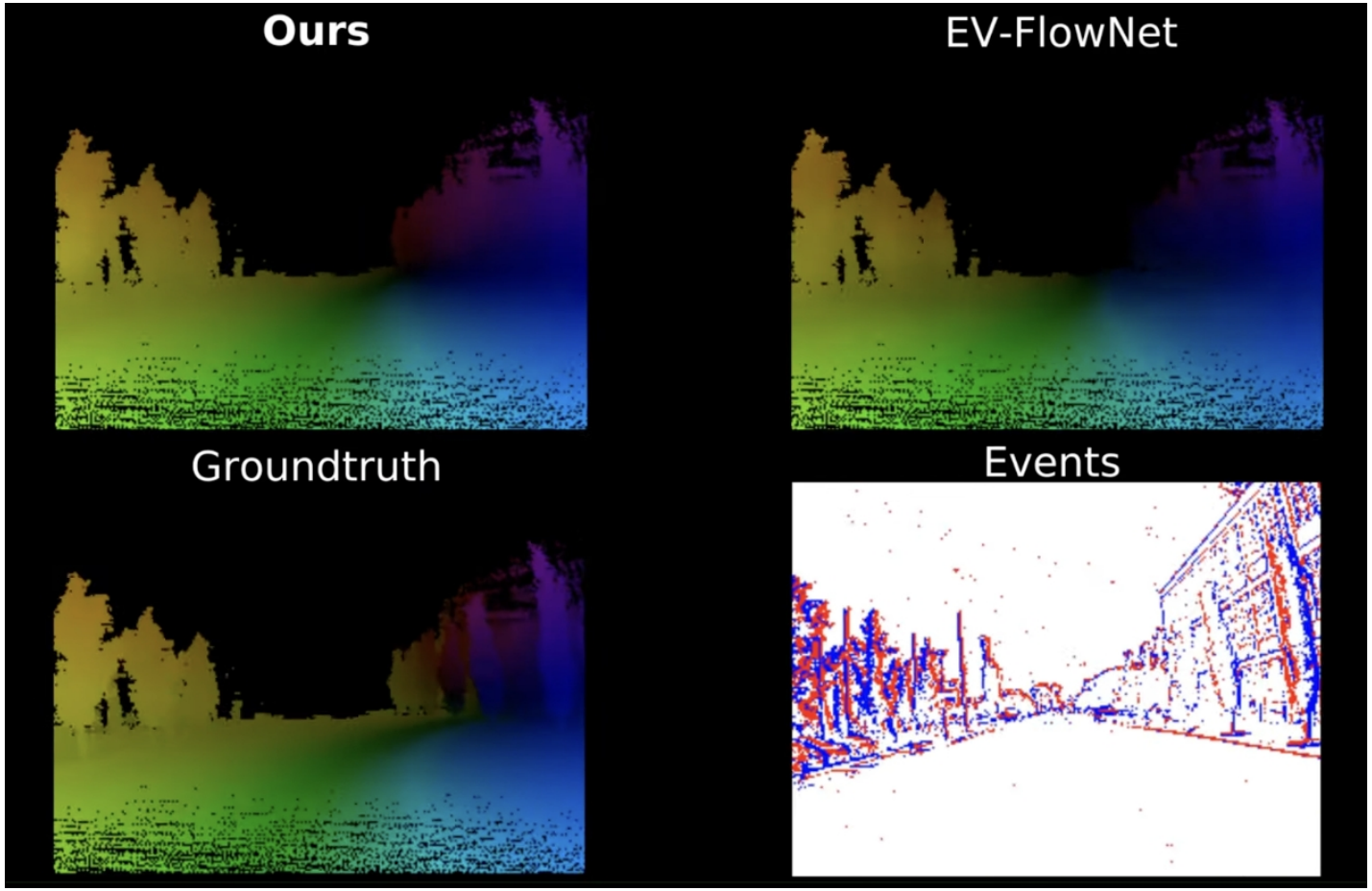
For the above two views for motion1/2, the fulcrum node is NOT in blade itself.

Skleton for Wind Turbine Failed !



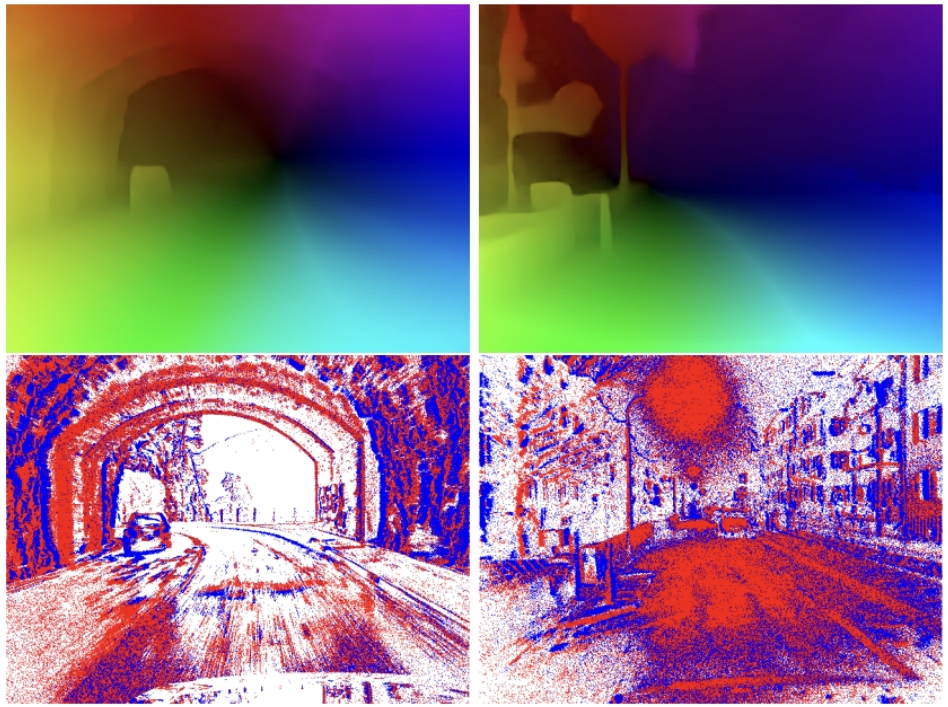
Node 1 / 2 / 3 are located in the connection point of the fan, not in the middle of the fan blades.

# Experiment 3: Dense Optical Flow from Event




https://github.com/DSaurus/MD-Splatting

Optical flow predictions from our proposed approach for event-based vision



Input: event data
Output: optical flow

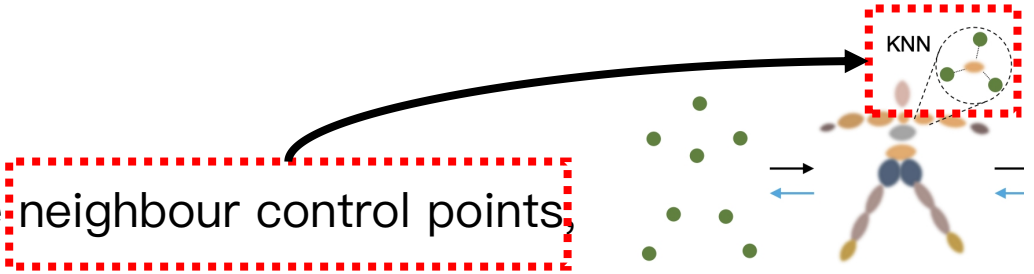
# Experiment 4: SC-GS: Sparse-Controlled GS

3D Gaussian Splatting + Dynamic Scenes + Editability Controls: While ensuring real-time rendering quality, it makes "dynamic 3D Gaussian scenes" controllable, editable, and layered.

Idea：Use sparse control points (Sparse Control Block = **Control Points**) to drive the surrounding 3DGs.

From Sparse → Dense:

> Each Gaussian points first find the neighbour control points,




Left: spase

Right: 3DGs points

– Dynamic Deformation: For each control point：input-t, output deformation field (dynamic motion traj)

> then they will be based on dist to learn weight, then combined with multiple controls.

# Experiment 4: SC-GS: Sparse-Controlled GS

Idea: Use sparse control points (Sparse Control Block = **Control Points**) to drive the surrounding 3DGs.

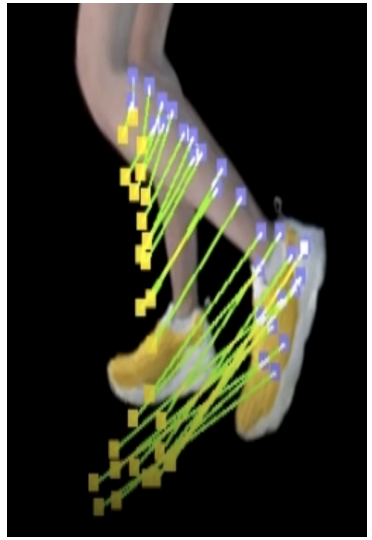


init: extract control points
driven **by KNN methods**



Sparse Control Points





Control Points Motion